

```
1 begin
2   using IntervalArithmetic
3   using Printf
4 end
```

Dict("Jupiter-Europa" $\Rightarrow$ Dict("L1" $\Rightarrow$ $[[0.979764, 0.979764]]_{256_trv_NG}$), "L2" $\Rightarrow$ $[[1.02046, 1.$

```

1 begin
2   setprecision(BigFloat, 256)
3
4   f_L1( $\xi$ ,  $\mu$ ) =  $\xi^5 - (3 - \mu)*\xi^4 + (3 - 2\mu)*\xi^3 - \mu*\xi^2 + 2\mu*\xi - \mu$ 
5   fd_L1( $\xi$ ,  $\mu$ ) =  $5\xi^4 - 4(3 - \mu)*\xi^3 + 3(3 - 2\mu)*\xi^2 - 2\mu*\xi + 2\mu$ 
6
7   f_L2( $\xi$ ,  $\mu$ ) =  $\xi^5 + (3 - \mu)*\xi^4 + (3 - 2\mu)*\xi^3 - \mu*\xi^2 - 2\mu*\xi - \mu$ 
8   fd_L2( $\xi$ ,  $\mu$ ) =  $5\xi^4 + 4(3 - \mu)*\xi^3 + 3(3 - 2\mu)*\xi^2 - 2\mu*\xi - 2\mu$ 
9
10  f_L3( $\xi$ ,  $\mu$ ) =  $\xi^5 + (2 + \mu)*\xi^4 + (1 + 2\mu)*\xi^3 + (\mu - 1)*\xi^2 + (2\mu - 2)*\xi + (\mu - 1)$ 
11  fd_L3( $\xi$ ,  $\mu$ ) =  $5\xi^4 + 4(2 + \mu)*\xi^3 + 3(1 + 2\mu)*\xi^2 + 2(\mu - 1)*\xi + (2\mu - 2)$ 
12
13  const ISOLATION_DIAM = BigFloat("1e-14")
14  const TARGET_DIAM    = BigFloat("1e-16")
15
16  function refine(X::Interval{BigFloat}, f, fd,  $\mu$ )
17    for _ in 1:400
18      diam(X) <= TARGET_DIAM && return X
19      fX = fd(X,  $\mu$ )
20      in_interval(0, fX) && return X
21      m = interval(mid(X), mid(X))
22      Y = intersect_interval(X, m - f(m,  $\mu$ ) / fX)
23      isempty_interval(Y) && return X
24      diam(Y) >= diam(X) * BigFloat("0.999") && return X
25      X = Y
26    end
27    return X
28  end
29
30  function find_roots!(X::Interval{BigFloat}, f, fd,  $\mu$ , solutions; depth=0)
31    depth > 80 && return
32    isempty_interval(X) && return
33    fX = fd(X,  $\mu$ )
34    if in_interval(0, fX)
35      m = (inf(X) + sup(X)) / 2
36      find_roots!(interval(inf(X), m), f, fd,  $\mu$ , solutions; depth=depth+1)
37      find_roots!(interval(m, sup(X)), f, fd,  $\mu$ , solutions; depth=depth+1)
38    end
39  end

```

```

39     end
40     m = interval(mid(X), mid(X))
41     Y = intersect_interval(X, m - f(m, μ) / fX)
42     isempty_interval(Y) && return
43     if diam(Y) < ISOLATION_DIAM
44         push!(solutions, Y)
45         return
46     end
47     if issubset_interval(Y, X) && diam(Y) < diam(X) * BigFloat("0.5")
48         find_roots!(Y, f, fd, μ, solutions; depth=depth+1)
49     else
50         m2 = (inf(X) + sup(X)) / 2
51         find_roots!(interval(inf(X), m2), f, fd, μ, solutions; depth=depth+1)
52         find_roots!(interval(m2, sup(X)), f, fd, μ, solutions; depth=depth+1)
53     end
54 end

55
56 systems = Dict(
57     "Sun-Jupiter"      => BigFloat(9.537e-4),
58     "Sun-(Earth+Moon)" => BigFloat(3.036e-6),
59     "Earth-Moon"       => BigFloat(1.215e-2),
60     "Mars-Phobos"      => BigFloat(1.667e-8),
61     "Jupiter-Io"       => BigFloat(4.704e-5),
62     "Jupiter-Europa"   => BigFloat(2.528e-5),
63     "Jupiter-Ganymede" => BigFloat(7.804e-5),
64     "Jupiter-Callisto" => BigFloat(5.667e-5),
65     "Saturn-Mimas"     => BigFloat(6.723e-8),
66     "Saturn-Titan"     => BigFloat(2.366e-4),
67     "Neptune-Triton"   => BigFloat(2.089e-4),
68     "Pluto-Charon"     => BigFloat(1.097e-1),
69 )
70
71 lagrange_config = [
72     ("L1", f_L1, fd_L1, BigFloat(0.0), BigFloat(1.0),
73      (μ, ξ) -> interval(1 - μ) - ξ),
74     ("L2", f_L2, fd_L2, BigFloat(0.0), BigFloat(1.0),
75      (μ, ξ) -> interval(1 - μ) + ξ),
76     ("L3", f_L3, fd_L3, BigFloat(0.5), BigFloat(1.5),
77      (μ, ξ) -> -interval(μ) - ξ),
78 ]

```

```

79
80     all_solutions = Dict()
81
82     for (sysname, μ) in systems
83         pts = Dict{String, Any}()
84         for (lname, f, fd, x0lo, x0hi, transform) in lagrange_config
85             sols = Interval{BigFloat}[]
86             find_roots!(interval(x0lo, x0hi), f, fd, μ, sols)
87             valid = Interval{BigFloat}[]
88             for ξ in sols
89                 inf(ξ) < -BigFloat("1e-10") && continue
90                 ξr = refine(ξ, f, fd, μ)
91                 push!(valid, transform(μ, ξr))
92             end
93             pts[lname] = valid
94         end
95         all_solutions[sysname] = pts
96     end
97
98     println("="^70)
99     for sysname in sort(collect(keys(all_solutions)))
100         println("\n$(sysname):")
101         for lname in ["L1", "L2", "L3"]
102             for iv in all_solutions[sysname][lname]
103                 lo = inf(iv)
104                 hi = sup(iv)
105                 @printf("  %s: [%.16f , %.16f]  width=%.3e\n",
106                     lname, Float64(lo), Float64(hi), Float64(hi - lo))
107             end
108         end
109     end
110     println("="^70)
111
112     all_solutions
113 end

```

Saturn-Phobos:

L1: [0.9971832458595825 , 0.9971832458595825] width=5.266e-30
L2: [1.0028219187888192 , 1.0028219187888192] width=1.480e-30
L3: [-1.0000000280125001 , -1.0000000280125001] width=5.299e-27

Saturn-Titan:

L1: [0.9574985190370510 , 0.9574985190370511] width=7.046e-17
L2: [1.0432540185142787 , 1.0432540185142787] width=1.759e-23
L3: [-1.0000985833326135 , -1.0000985833326135] width=3.288e-26

Sun-(Earth+Moon):

L1: [0.9899908275855027 , 0.9899908275855027] width=3.157e-19
L2: [1.0100702983726619 , 1.0100702983726619] width=6.349e-21
L3: [-1.0000012650000001 , -1.0000012650000001] width=1.560e-25

Sun-Jupiter:

L1: [0.9323697524160933 , 0.9323697524160933] width=2.501e-18
L2: [1.0688263265633298 , 1.0688263265633298] width=7.220e-22